

Translating the ARM Neon and VFP instructions in a binary translator

Yu-Chuan Guo¹, Wu Yang^{1,*}, Jiunn-Yeu Chen¹ and Jenq-Kuen Lee²

¹National Chiao-Tung University, Hsinchu, Taiwan

²National Tsing Hua University, Hsinchu, Taiwan

SUMMARY

Binary translation attempts to emulate one instruction set with another on the same or different platforms. The important technique is widely used in modern software. Vector and floating-point instructions are widely used in many applications, including multimedia, graphics, and gaming. Although these instructions are usually simulated with software in a binary translator, it is important to support them such that the host single-instruction, multiple-data (SIMD) and floating-point hardware are efficiently used during emulation. We report our design and implementation of the emulation of ARM Neon and vector floating point (VFP) instructions in the machine-code-to-low-level-virtual-machine (MC2LLVM) binary translator. The Neon and VFP instructions are first translated into carefully chosen sequences of LLVM intermediate representation (IR), and later, the IR sequences are optimized and translated into the host native binary by the existing LLVM backend. Because MC2LLVM makes use of the vector and floating-point types in LLVM IR, the generated host native binary can take full advantage of the vector and floating-point functional units, if present, of the host machine. To be fully compliant with Neon and VFP instruction sets, all the features are supported, including the flush-to-zero mode, default not a number mode, and floating-point exceptions. The experimental results show that code generated by MC2LLVM with the Neon and VFP extensions achieves an average speedup of 1.174 $\times$ in SPEC 2006 benchmark suites and exhibits a floating-point throughput of 12.05 $\times$ in LINPACK, compared with code generated by MC2LLVM without the Neon and VFP extensions. Furthermore, MC2LLVM is 3.36 $\times$ faster than QEMU for processing Neon/VFP instructions. Copyright © 2016 John Wiley & Sons, Ltd.

Received 27 April 2015; Revised 15 January 2016; Accepted 15 January 2016

KEY WORDS: binary translation; cloud computing; LLVM; floating-point instruction; Neon; vector instruction; VFP; virtualization

1. INTRODUCTION

Binary translation [1] attempts to emulate one instruction set with another on the same or different platforms. For example, an ARM program can be executed on an x86 platform with the help of a binary translator. The important technique is widely used in instruction-set-architecture migration [2–6], binary instrumentation [7–9], dynamic optimizations [10–13], software security [14], and fast architecture simulation [15–19]. The Neon and VFP coprocessors [20] are extensions to the ARM architecture. They are designed for applications with SIMD and floating-point instructions to meet the growing demand of computing power in embedded systems, such as multimedia, graphics, and gaming.

Several existing binary translators, such as QEMU, support vector and floating-point instructions with naive software simulation (using scalar, integer, and shift operations) [21]. The result is poor performance. In this paper, we attempt to make use of the vector and floating-point hardware on the host platform, if present, to execute the Neon and VFP instructions. Furthermore,

*Correspondence to: Wu Yang, National Chiao-Tung University, Hsinchu, Taiwan.

†E-mail: wuyang@cs.nctu.edu.tw

all the related exceptions and flags in the floating-point environment are taken care of properly. Our system is built on top of MC2LLVM a retargetable static/dynamic/hybrid binary translator developed in our lab in the past few years. The resulting performance is the key issue in our design.

To build a retargetable binary translator, a two-stage translation is adopted, which partitions the translation process into target-independent (frontend) and target-dependent (backend) stages. We leverage the existing LLVM backend to build a high-performance and retargetable binary translator. In this research, the emulation of Neon and VFP architectures is layered on the top of an LLVM backend.

To be fully compliant with Neon and VFP instruction set architectures, we need to know the details of the Neon and VFP architectures [20], including the flush-to-zero mode, the default NaN (not a number) mode, and the floating-point exceptions. These details can affect the behaviors of the Neon and VFP instructions. Our implementation must faithfully mimic these features. We also propose new methods to detect floating-point exceptions if the IR layer of a binary translator does not provide the relevant information.

To achieve high performance, we invest much effort to enhance the quality of the translated code. The emulation of each instruction is carefully hand optimized. If an instruction cannot be emulated with a few LLVM IRs, there are usually several algorithms for emulation. We evaluated various emulation methods of an instruction with several benchmarks in order to choose the best.

To achieve a high degree of reliability, we also developed a verification framework for testing all the emulated Neon/VFP instructions. Verification is performed automatically.

The remainder of this paper is organized as follows: Section 2 lists the background and related work, including Neon and VFP architectures, and describes the related terminology. In Section 3, we describe the implementation details. Section 4 illustrates the verification framework. Section 5 discusses the experimental results. Section 6 concludes this paper.

2. BACKGROUND AND RELATED WORKS

In this research, we attempt to translate the Neon and VFP instructions into LLVM IR. In this section, we will introduce the background and related work about the Neon and VFP architectures, including machine features related to binary translation.

2.1. Low Level Virtual Machine

Low Level Virtual Machine [22] is an open-source compiler infrastructure designed as a set of libraries. It provides the middle layers of a compiler system around a well-defined intermediate representation, known as LLVM intermediate representation (IR). LLVM IR may be compiled to machine code by a static compiler or a just-in-time (JIT) compiler. In the past decade, LLVM continues to evolve to improve its functionality, robustness, and advanced optimizations. Developing a binary translator on top of LLVM takes advantage of these features and significantly reduces development time. However, one of the most important features that LLVM lacks today is the support for the access of the floating-point environment.

The LLVM IR is a virtual instruction set in the static single assignment (SSA) form [23], with a strong type system and polymorphic instructions. It supports operations that use vector and floating-point data types, so we could emulate the Neon and VFP instructions without losing instruction-level parallelism and the essence of floating-point operations in the frontend translation – ARM to LLVM IR – in our binary translator MC2LLVM.

MC2LLVM depends on the LLVM code generator to produce host SIMD instructions and floating-point instructions. Before instruction emulation, it is important to know the instruction patterns generated by the LLVM code generator from LLVM vector and floating-point operations in order to allow us to choose the best translation of each instruction. Our investigation in the LLVM x86 backend on Intel i7-4770 shows that it is capable of translating vector and floating-point operations in LLVM IR into advanced vector extensions (AVX) [24] instructions, which is an extension to x86 instruction set architecture and is designed for applications with SIMD operations and/or intensive floating-point operations.

2.2. MC2LLVM

MC2LLVM is a process-level binary translator based on LLVM developed in our lab in the past few years. It adopts the modern compilation approach that separates the translation process into a frontend and a backend. The frontend translates the guest binary into LLVM IR and then uses the existing LLVM backend to generate the host binary from LLVM IR.

MC2LLVM supports static, dynamic, and hybrid binary translation. A static binary translator (SBT) translates a binary file offline and thus can perform aggressive optimizations. But an SBT is confronted with several intractable problems, including code discovery, code location, and self-modifying code. On the other hand, a dynamic binary translator (DBT) translates a binary file at run time. DBT can handle the aforementioned problems more effectively. More runtime overhead, more memory usage, and the lack of optimizations, however, are DBT's main shortcomings.

To combine the merits of SBT and DBT, a hybrid binary translator translates a binary file offline first, and then, at run time, when an exception occurs, such as failing to find a branch target address, it switches to the DBT mode to handle the exception. It may switch back to statically translated binary later.

Figure 1 shows the flow of dynamic binary translation in MC2LLVM. The emulation manager maintains and manipulates the progress of the emulation, such as handling the control transfer between translated basic blocks in code cache and invoking the translator to translate a target basic block that has not yet been translated. In this research, we will focus on the translator module, which is also employed in the static and hybrid modes.

2.3. QEMU

QEMU [21] is an open-source dynamic binary translator, which supports full system emulation. QEMU has its own IR, known as tiny code generator (TCG) IR [21], to implement a two-stage translation. It is able to emulate several ISAs, such as x86, PowerPC, ARM, and MIPS, *etc.* It can also emulate the Neon and VFP instructions, but all floating-point operations are emulated in software and all vector operations are emulated as scalar operations. In contrast, our binary translator is able to make use of host vector/floating-point instructions, and it fully emulates these instructions, including condition flags and exceptions.

2.4. Neon and VFP Extensions

Neon is a single instruction multiple data (SIMD) processor integrated into the ARM chip. Neon provides a 64/128-bit SIMD instruction set that provides acceleration for multimedia and signal

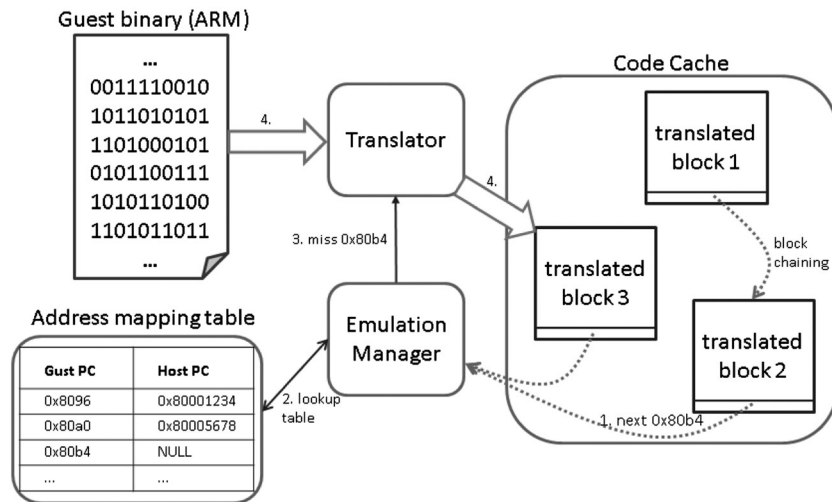


Figure 1. Dynamic binary translator mode of MC2LLVM.

processing applications, such as compressed video decoding, image processing, 2D/3D graphics, and sound synthesis. In Neon, registers are considered as vectors of elements of the same data types, which can be 8-bit, 16-bit, 32-bit, 64-bit signed/unsigned integers or single-precision floating-point numbers.

VFP is a vector floating-point processor integrated as a part of the ARM chip. The VFP architecture initially aimed at supporting vector operations, but the vector mode was later removed, to be replaced with Neon. VFP provides a single-precision and double-precision floating-point instruction set that is fully compliant with IEEE 754 [25]. (Neon is not fully compliant with IEEE 754.) There are several versions of VFP architectures. In this research, we will focus on VFPv3 (or VFPv3-D32).

Neon and VFP share the same extension register bank, which is distinct from the ARM core register bank. The register bank can be referred to in three aliased views. Figure 2 shows the common register bank in the single-word, double-word, and quad-word views. Neon can view the register bank as 32 double-word registers (i.e., D0-D31), 16 quad-word registers (i.e., Q0-Q15), or a combination of the aforementioned two views. VFP can see the register bank as 32 single-word registers (i.e., S0-S31; only a half of the register bank is accessible in this view), 32 double-word registers (i.e., D0-D31), or a combination of the aforementioned two views.

The Neon and VFP extensions have a shared register space for system registers. Only the system register known as floating-point status and control register (FPSCR) in this space is accessible in the application programs. FPSCR contains the arithmetic status flags as well as the bit fields for controlling the floating-point unit. Table I shows the bit fields of the FPSCR register.

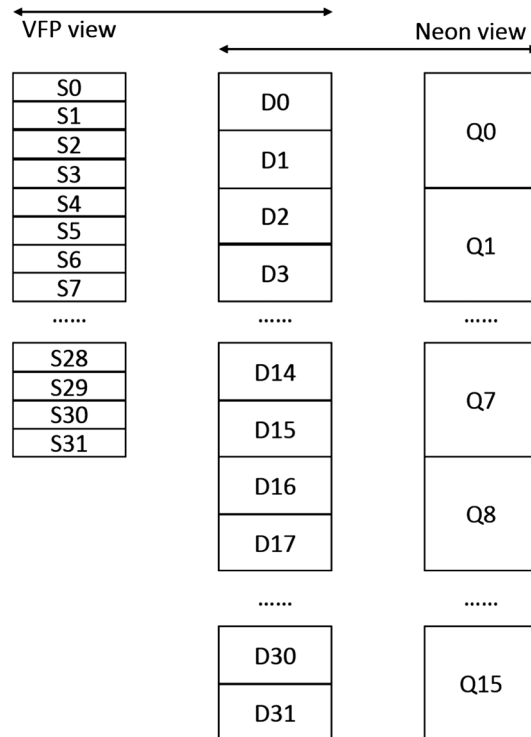


Figure 2. Neon and VFP extension register banks, which is adapted from [20].

Table I. Floating-point status and control register format.

Bit(s)	31	30	29	28	27	26	25	24	
Meaning	N	Z	C	V	QC	AHP	DN	FZ	
Bit(s)	23:22	21:8	7	6:5	4	3	2	1	0
Meaning	RMode	Reserved	IDC	Reserved	IXC	UFC	OFC	DZC	IOC

The N, Z, C, and V bits are negative, zero, carry, and overflow (or unordered) condition flags, respectively. They are updated by floating-point comparison operations such as `vcmp`. These flags are similar to the N, Z, C, and V flags in application program status register (APSR), but they do not directly control conditional execution. They must be copied into (by the instruction `vmrs apsr_nzcv, fpscr`) APSR in order to affect program execution.

The QC bit is the cumulative saturation flag. This flag is set to 1 if the result of a saturating instruction is saturated. This bit is updated by the Neon saturating instructions (described in a later section).

The AHP bit controls the alternative half-precision extension. The half-precision extension is an optional architecture that extends the Neon and VFP architectures. In this research, this alternative half-precision extension is not implemented, however.

The DN bit controls the default NaN (not a number) mode, which affects the behavior of the floating-point operations involving one or more NaNs. The NaN processing follows the IEEE 754 standard if the DN bit is 0. A floating-point operation involving one or more NaNs returns the default NaN if the DN bit is 1. This bit only affects the VFP instructions. Neon always uses the default NaN setting, regardless of the DN bit.

The FZ bit controls the flush-to-zero mode, which affects the behavior of the floating-point operations. If the FZ bit is 0, the behavior of a floating-point operation in VFP follows the IEEE 754 standard. If the FZ bit is 1, the flush-to-zero mode is enabled. In the flush-to-zero mode, denormalized numbers (in the IEEE 754 standard [25], denormalized or denormal numbers are very small numbers whose exponent fields are 0. They are used to fill the gap between zero and the minimum normalized number) will be flushed to 0. The flush-to-zero mode also changes the criteria for the floating-point exceptions to occur (described in a later section). Neon always uses the flush-to-zero mode, regardless of the value of the FZ bit.

The RMode field controls the rounding mode. There are four rounding modes: round to nearest (RN), round towards plus infinity (RP), round towards minus infinity (RM), and round towards zero (RZ). The RMode field only affects the VFP instructions. Neon always uses the round-to-nearest mode, regardless of the value of the RMode field.

IDC, IXC, UFC, OFC, DZC, and IOC are input denormal, inexact, underflow, overflow, division by zero, and invalid operation cumulative exception flags, respectively. These flags show abnormalities during floating-point operations. A *cumulative* exception bit is set to 1 when the corresponding floating-point exception occurs. However, it is not reset to 0 automatically when the corresponding exception does not occur. Applications can read these flags (including the QC flag) with instructions such as `vmrs r0, fpscr` (to copy FPSCR to r0) or can clear them by writing zero to them using instructions such as `vmsr fpscr, r0` (to copy r0 to FPSCR). These flags are usually used in applications with high safety requirements.

Taking advantage of the SIMD capability, such as Neon/VFP, is important in media applications. Jantz *et al.* [26] consider re-ordering the optimization phases for generating SIMD instructions. Pryanishnikov *et al.* [27] use interprocedural analysis to produce SIMD code that access data aligned properly in order to increase the performance of the resulting SIMD program. As SIMD applications become more and more popular, new libraries, such as Vc [28], appear, which generate portable and effective vectorized code. There are many applications of the SIMD instructions provided by Neon/VFP. Lemirel *et al.* [29] use SIMD vectorization for compression and decompression for database systems. Inoue *et al.* [30] demonstrate a high-performance sorting algorithm for multicore SIMD processors.

3. DESIGN AND IMPLEMENTATION

The emulation of the Neon and VFP extensions, including the Neon and VFP registers, instructions, and machine features, is discussed in this section.

Figure 3 shows the translation flow of MC2LLVM. The translator uses LLVM MCDisassembler [31] to disassemble binary instructions into MCInst (the IR of MCDisassembler). Each MCInst is translated by the corresponding translation function into LLVM IR. The LLVM IR is organized

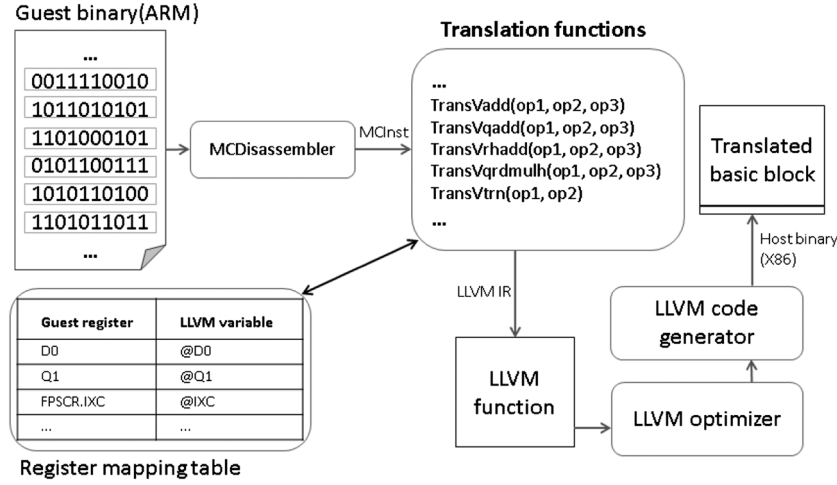


Figure 3. Translation from guest binary to host binary.

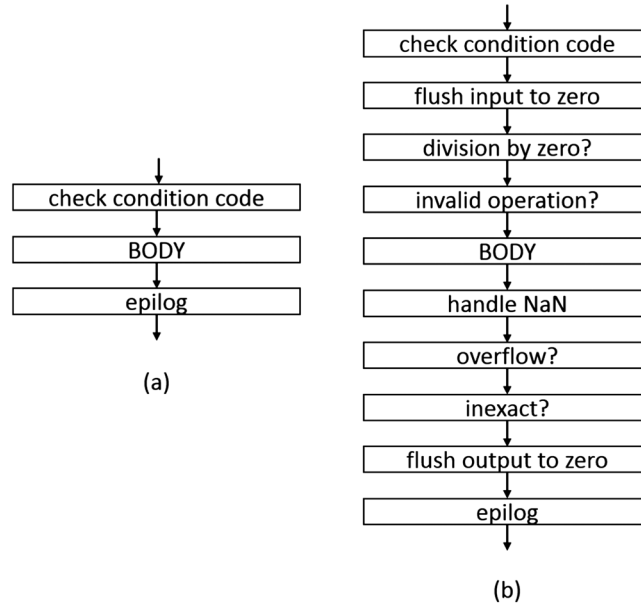


Figure 4. Components in a translation function: (a) a translation function that does not involve the floating-point environment and (b) the refined translation function for the floating-point environment.

as LLVM functions. The LLVM optimizer performs target-independent optimizations before code generation.

Previously, MC2LLVM only supports ARMv5 and Thumb-2 [20] ISAs. Because ARMv5 and Thumb-2 instructions are relatively simple, a translation function consists of three steps: (i) the prolog checks the condition code; (ii) the body emulates the given operation; and (iii) the epilog branches to the next translated instruction, as shown in Figure 4(a). However, the Neon and VFP instructions are more complex because of the floating-point environment. The translation function for an instruction is further refined, as shown in Figure 4(b).

3.1. Register emulation

The user-level registers in Neon and VFP are emulated with LLVM global variables. The extension register bank is emulated with a 256-byte LLVM global array initialized with zero:

```
@regBank = internal global [256 x i8] zeroinitializer, align 256
```

This array is pointed to by LLVM global pointers, which serve as the Neon and VFP registers. Figure 5 shows the emulated Neon and VFP register set. For easy access to the fields in FPSCR, they are individually emulated by LLVM global 32-bit integer variables. Instructions that transfer values between ARM core registers and FPSCR such as `vmsr fpscr, r0` will get a bit more performance penalty in scattering/gathering.

To implement the data types supported by Neon and VFP, we convert the value of an emulated register to another type with the LLVM `bitcast` instruction, which casts a value to a different type without changing any bits.

3.2. Instruction emulation

In our research, all the instructions in Neon and VFP are emulated with LLVM IRs and LLVM intrinsic functions. There are 1244 instructions in the Neon and VFPv3 instruction sets among which only four instructions that involve half-precision floating-point operations have not been implemented yet. For example, Figure 6 shows the emulation of the VFP multiply instruction in MC2LLVM. The

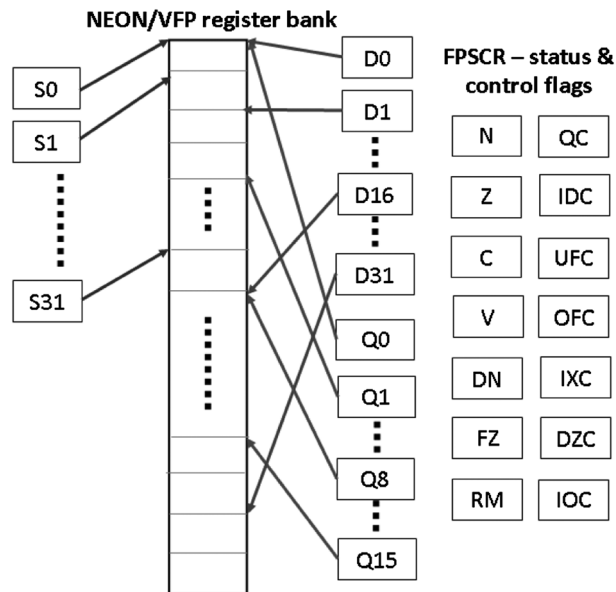


Figure 5. Emulation of Neon and VFP extension registers and exception flags.

```
VFP instruction:
vmul.f64 d0, d1, d2
Translated to LLVM IR:
// loads the operands
%1 = bitcast i64** @D1 to double**
%2 = load double** %1
%3 = load double* %2
%4 = bitcast i64** @D2 to double**
%5 = load double** %4
%6 = load double* %5
// executes the body
%7 = fmul double %3, %6
// stores the result
%8 = bitcast i64** @D0 to double**
%9 = load double** %8
store double %7, double* %9
```

Figure 6. Emulation of the VFP multiply instruction.

Table II. Instruction modifiers.

Instruction set	Category	Modifier	Example
Neon	Load-and-compare		vand q3, q5, q6 vbic.i16 q1, #0x0F vacge.f32 q0, q10
	Data processing		vcvt.s32.f32 q0, q1, #12 vzip.32 q0, q3
		Q	vqmovn.u64 d0, q1
		N	vqmovn.s32 d2, q3
		U	vqmovun.s64 d1, q2
	Shift		vshl.s32 q1, q2 vsli.8 q0, #3
		Q	vqshl.u8 q0, q1
		R	vqrshl.s16 q1, q2
		N	vqrshrn.u32 d0, q1, #5
		U	vqrshrun.s64 d0, q2, #33
	Arithmetic		vadd.i32 q1, q2, q3 vrecpe.f32 d0, d1
		Q	vqadd.s8 d1, d2, d3
		R	vqrdmulh.s16 q0, q1
		D	vqrdmulh.s16 q1, q2, d0[3]
		H	vqrdmulh.s32 q0, q2
			vrhadd.u32 d0, d1, d2
		P	vpadd.s16 d0, d1
		N	vaddhn.i32 d0, q1, q2
		L	vaddl.s32 q1, d3, d4
		W	vaddw.s16 q1, q2, d8
VFP	Load-store elements/structure		vld1.16 {d0[2]}, [r0], r1 vld2.8 {d1[], d2[]}, [r3]! vld3.32 {d0, d2, d4}, [r2]!
	Data processing		vadd.f64 d0, d1, d2 vmul.f32 s0, s1, s2 vcvt.f32.f64 s0, d2 vcmp.e.f32 s20, s31
	Load and store		vldr d1, [pc, #-8] vstr s1, [r1, #4] vldmdb r2, {s4-s7} vstmia sp!, {d0, d7}
Neon and VFP commons	Data transfer between core and registers		vmov s0, s1, r2, r6 vmsr fpscr, r0 vmrs r0, fpscr vmrs apsr_nzcv, fpscr

operands (in the D1 and D2 registers) are loaded into the local variables %3 and %6, respectively, with appropriate type casting. Then the floating-point multiplication `fmul` is performed. Finally, the result is stored into register D0. Floating-point exceptions, the flush-to-zero mode, and the default NaN mode will be discussed in later sections.

Table II categorizes the Neon and VFP instructions. There are several classes of instructions in Neon prefixed and/or postfixed with a combination of the following modifiers, shown in Table III and with various addressing modes. Please see ARM Architecture Reference Manual [20] for details.

The LLVM IR provides very general and primitive IRs for logical (e.g., `and`, `or`), arithmetic (e.g., `add`, `mul`), shift (e.g., `shl`, `lshr`), and load/store (e.g., `load`, `store`) instructions and various data types (e.g., vector of 4 32-bit integers, `<4 x i32>`; vector of 2 floating-point numbers, `<2 x float>`) to allow emulating the majority of Neon instructions with just a few IRs. For modifiers

Table III. Instruction modifiers.

Q	saturating instruction (the instructions will be discussed later)
R	round half away from zero
D	doubling the size of the result
H	halving the size of the result or returning the high half of the result
P	pairwise, operating on adjacent pairs of elements of two input vectors
N	narrow instruction, the width of the elements of the result is a half of the width of the elements of its input operands
L	long instruction, the elements of the result are twice as wide as the elements of its input operands
W	wide instruction, the elements of the result and first input operand are twice as wide as the elements of second input operand

```

1.  vqshl.s32(op1, op2)
2.    if(op2 > 0)
3.        result = op1 << op2;
4.        mask = (signed) 0x80000000 >> op2;
5.        if(op1 > 0)
6.            if((mask & op1) != 0) // if overflow
7.                SetQC();
8.            result = 0x7FFFFFFF;
9.        if(op1 < 0) // to test if underflow
10.            if((mask & op1) != mask && op2 >= 32)
11.                SetQC();
12.            result = 0x80000000;
13.        else result = op1 >> op2;
14.        return result;

```

Figure 7. Algorithm for signed saturating shift left instruction.

of Neon instructions, N can be emulated with LLVM `trunc` (truncation) instruction, L and W with LLVM `sext` (signed extension) or `zext` (zero extension) instructions, and P with LLVM `shufflevector` instruction to reorder the elements of input operands. The Neon floating-point and saturating instructions may be the hardest to emulate with LLVM IRs. We will discuss saturating instructions in the next section. A complete discussion of floating-point instructions together with related exceptions will be discussed in a later section.

3.2.1. Saturating instruction emulation. Neon saturating instructions are integer operations, which replace an overflow result with the maximum value or an underflow result with the minimum value. Saturating instructions also set the cumulative saturation flag, QC, to 1, when overflow or underflow occurs.

Because LLVM does not provide saturating operations, emulation of saturating instructions would impose extra overhead (including detecting integer overflow and underflow and setting QC). There are usually several ways to emulate a saturating instruction.

For example, the `vqshl` (vector saturating and shift left) instruction, one of the 155 saturating instructions in Neon, takes each element in a first vector, shifts it by the value taken from the least significant byte of the corresponding element in a second vector, and places the results in the destination vector. The operation can be shift left or shift right depending on the sign of the shift value. However, there are separate shift instructions for left shift (`shl`) and right shift (`lshr`) in LLVM IR.

Furthermore, if its shift value is greater than or equal to the number of bits in the shifted value, the result is undefined. These limitations make it hard to emulate `vqshl` correctly and efficiently. Figure 7 shows an algorithm for emulating `vqshl` with the 32-bit signed integer type. Only for a shift-left could saturation happen. If the operand to be shifted, say `op1`, is positive and any non-zero bits will be shifted through signed bit (tested by a mask), an overflow occurs. Similarly, if `op1` is negative and any zero bits will be shifted through signed bit, an underflow occurs.

Notice that the algorithm in Figure 7 is assumed to be executed for all elements of a vector *in parallel*. In fact, the majority of the translations of the Neon instructions are assumed to be executed

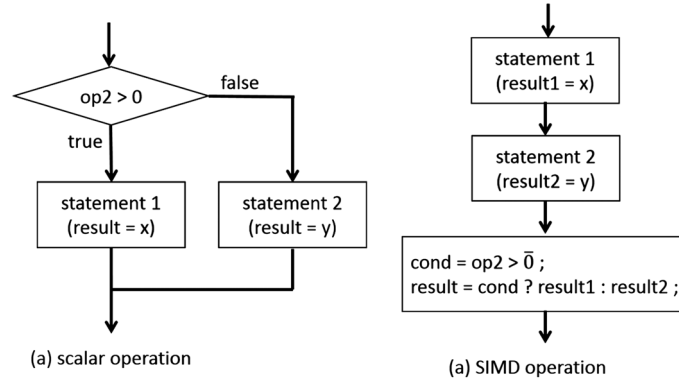
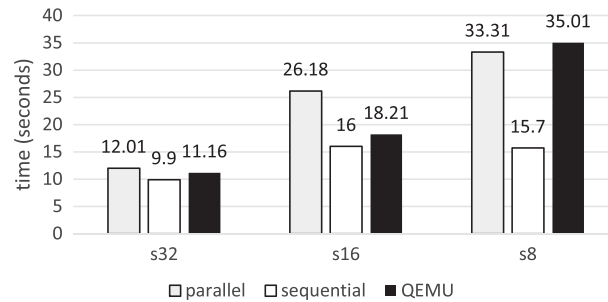


Figure 8. Branch divergence problem in SIMD operations.

```

// init:  r0 = 0; r2 = 1,000,000,000
LOOP:
add r0, #1
vqshl.s32 q0, q1, q2 // body
subs r3, r2, r0
bne LOOP

```

Figure 9. A benchmark for evaluating the emulation of `vqshl`.Figure 10. Execution time of `vqshl` running on MC2LLVM and QEMU, respectively.

in parallel. This presents a problem similar to the branch divergence problem in GPUs. Figure 8 illustrates a conditional branch implemented with scalar operations and with SIMD operations, respectively. In SIMD operations, unlike scalar operations, all the paths must be traversed in order to obtain the results of each conditional branch. Conditional branches with SIMD operations diminish the efficiency of parallel emulation, and, if conditional branches are nested deeply, the sequential implementation may even outperform the parallel implementation.

To evaluate the performance of the translated instructions, we wrote a few benchmarks in ARM assembly. An example is shown in Figure 9, in which we execute the instruction `vqshl.s32 q0, q1, q2` one billion times. Figure 10 shows the average execution time of the benchmarks with the types `s32`, `s16`, and `s8` on MC2LLVM and QEMU, respectively. The logos `parallel` and `sequential` stand for parallel and sequential implementations, respectively, of the algorithm in Figure 7 running on MC2LLVM. According to our experiments, all the floating-point operations that involve many `if-else` checks should be implemented in the sequential fashion. Most integer instructions are suitable for a parallel implementation except `vqshl` and `vqsrhl` (vector saturating rounding shift left with signed shift values), which are implemented in a sequential fashion in our emulated Neon and VFP extensions.

```

1. procedure FP32_FlushInputToZero(operand)
2.   if (IsDenormal(operand))
3.     SetIDC(); // input denormal exception
4.     flush operand;
5.
6. function IsDenormal(op)
7.   if (fabs(op) < 0x00800000 && fabs(op) > 0)
8.     return TRUE;
9.   return FALSE;

```

Figure 11. Algorithm for flush-input-to-zero.

3.3. Flush-to-zero mode emulation

The flush-to-zero mode [20] is a special processing mode that replaces denormalized operands, intermediate results, and final results with zero while reserving the sign bit. It is used to avoid handling denormalized numbers, thus saving execution time. Because supports for denormalized numbers increase the complexity in hardware design [32], floating-point units often save hardware cost by simply delegating the supports for denormalized numbers to software.

The flush-to-zero mode is not compliant with IEEE 754. It changes the behavior of a floating-point operation and the criteria for the floating-point exceptions in three ways: (i) In the flush-to-zero mode, a floating-point operation can cause the input denormal exception, which is not included in IEEE 754. (ii) The inexact exception would not be raised when a result is rounded to zero or flushed to zero. (iii) If the result of an operation is rounded to zero or flushed to zero, the underflow exception would be raised.

Because the LLVM IR does not provide the flush-to-zero operation, the flush-to-zero mode in our system is emulated in software. There are two operations in the flush-to-zero mode: flush-input-to-zero and flush-output-to-zero. Each floating-point operation must go through the preprocessing in LLVM IR in which denormalized numbers are detected and replaced with zero before a floating-point operation is performed. This is the flush-input-to-zero operation, shown in Figure 11. The emulation of the `vmul.f32` instruction in Figure 12 shows an example of flush-input-to-zero in the block marked as `FZ_BB`.

The encoding of a floating-point number is treated as integer in the flush-to-zero processing. According to IEEE 754, the representation of the absolute value of a denormalized single-precision floating-point number is between `0x00000001` and `0x007FFFFF` (the exponent field is all 0s and the fraction field is not all 0s). A double-precision floating-point number or a vector is processed similarly.

According to the definition of the flush-to-zero mode, if the intermediate result of a floating-point operation that is produced *before* rounding satisfies the following condition:

$$0 < \text{abs}(\text{result}) < +\text{MinNorm} \quad (1)$$

where *MinNorm* is the minimum normalized number of the destination precision, that intermediate result is flushed to zero before the rest of the floating-point operation. However, an LLVM floating-point operation can only produce a result *after* rounding. The intermediate result produced after the floating-point operation but before rounding is invisible. Therefore, the flush-to-zero operation may produce an incorrect result if the intermediate result has already been rounded to *MinNorm* or to zero. Hence the flush-output-to-zero operation, shown in Figure 13, is introduced to flush the intermediate and final results of a single-precision multiplication to zero.

If both operands of a floating-point multiplication are not zero but the result is zero, the result must have been rounded to zero. Obviously, the multiplication causes an underflow exception. Only when the intermediate result that is produced before rounding satisfies the following condition:

$$+\text{MaxDenorm} < \text{abs}(\text{result}) < +\text{MinNorm} \quad (2)$$

where *MaxDenorm* is the maximum denormalized number (that is, the exponent field is all 0s and the fraction field is all 1s) of the destination precision, the result could be rounded to *MaxDenorm* or *MinNorm*. In this case, the after-rounding result may be a normalized number, but it should be

```

VFP instruction:
vmul.f32 s0, s1, s2

Translated to LLVM IR:
// ignores the part of loading operands
.....
BB_0:
// check if the flush-to-zero mode is activated
%1 = load i32* @FZ
%isFZmode = trunc i32 %1 to i1
br i1 %isFZmode, label %FZ_BB, label %BB_1
FZ_BB:
// flushes s1 (or %op1) to zero preserving the
// sign bit if s1 is denormal;
%2 = bitcast float %op1 to i32
%3 = and i32 0x7FFFFFFF, %2
%4 = icmp ult i32 %3, 0x800000
%5 = icmp ne i32 %3, 0
%6 = and i1 %4, %5
%7 = and i32 %2, 0x80000000
%8 = bitcast i32 %7 to float
%9 = select i1 %6, float %8, float %op1
// raises an input denormal exception
%10 = load i32* @IDC
%11 = zext i1 %6 to i32
%newIDC = or i32 %11, %10
store i32 %newIDC, i32* @IDC
br label %BB_1
BB_1:
// selects the flushed s1 or the original s1
// depending on FZ bit
%new_op1 = phi i32 [%op1, %BB_0], [%9, %FZ_BB]

```

Figure 12. Emulation of the VFP single-precision multiply instruction.

```

1. procedure FP32Mul_FlushOutputToZero(op1, op2, result)
2.   if (isDenormal(result))
3.     SetUFC(); // underflow exception
4.     flush result;
5.   else if (isZero(result) && !isZero(op1) && !isZero(op2))
6.     SetUFC(); // underflow exception
7.   else if (isMinNorm(result))
8.     A = multiply op1's and op2's fractions with leading
1's;
9.     if(A has 23 consecutive 1s starting at the most
significant 1)
10.      SetUFC();
11.      flush result;

```

Figure 13. Algorithm for flush-output-to-zero for a single-precision floating-point multiplication.

flushed to zero. We can reproduce the before-rounding intermediate result by (integer-)multiplying the two operands' fractions. If the intermediate result has 23 (if single precision) consecutive 1's starting from the most significant 1, that is, condition (2), the result should be flushed to zero and an underflow exception should be raised.

Different floating-point instructions come with different emulations of the flush-output-to-zero operation. For example, for floating-point additions, it is not necessary to consider the two special cases (rounding to zero or to *MinNorm*) because a floating-point addition is always exact (i.e., without loss of precision) when the result is a denormalized number.

```

1. procedure FP32.NaNHandling(result, isInvalidOp)
2.     // isInvalidOp is true if invalid operation
3.     // exception occurs
4.     // getDN(): get the DN bit in FPSCR
5.     if ((IsNaN(result) && getDN()) || IsInvalidOp)
6.         result = 0x7fc00000; // default NaN

```

Figure 14. Emulation of the default not a number (NaN) mode.

3.4. Emulation of the default NaN mode

In the IEEE 754 standard, *NaN* is used to represent a value that is actually not a real number, for example, $\text{inf} + (-\text{inf})$, $\text{inf} \times 0$, and $\text{sqrt}(-2)$. NaN is encoded such that the exponent field is all 1s and the fraction field is not all 0s. There are two kinds of NaNs: *signaling NaNs* (SNaNs, the most significant fraction bit is 0) and *quiet NaNs* (QNaNs, the most significant fraction bit is 1). SNaNs will raise an invalid operation exception when a floating-point operation is applied on them. QNaNs propagate errors resulting from invalid operations without raising exceptions. There are many encodings of SNaN and QNaN. In the Neon and VFP architectures, there is a special QNaN, called the *default NaN*, which is encoded as 0x7FC00000 in single precision and 0x7FF8000000000000 in double precision. (In x86, the sign bit of the default NaN depends on the operands but, in Neon and VFPv3, the sign bit of the default NaN is always 0.)

In VFPv3, there are two ways to handle a NaN depending on the default NaN mode control bit in FPSCR (in contrast, Neon always uses the enabled default NaN mode). When the default NaN mode is disabled

1. If an invalid operation exception is raised because of an operation involving one or more signaling NaNs, the operation will return the quiet NaN, which is equal to the first signaling NaN (from left to right) with its most significant fraction bit changed to 1.
2. If an invalid operation exception is raised for other reasons, it will return the default NaN (which is also a quiet NaN).
3. If an operation involving one or more quiet NaNs is performed, it will return the first quiet NaN.

When the default NaN mode is enabled, an operation that should return a NaN in the above cases returns the default NaN. The default NaN mode in our translation system is emulated in software, as shown in Figure 14.

3.5. Floating-point exception emulation

Floating-point exceptions may be emulated with two methods. The first method checks the exception flags in the underlying hardware (based on C++11 Standard Library), and the second method employs additional test code. The second method is complete and more efficient than the first. We will explain both methods in this section.

Although LLVM supports many floating-point operations, it does not provide any information related to floating-point exceptions. The developers of LLVM may consider that accessing the floating-point environment is unlikely to happen and supporting them would diminish the performance because of the implicit data dependencies that might occur in the floating-point environment. For example, setting floating-point exception flags or trapping the floating-point exceptions for further processing may unnecessarily restrict the LLVM optimizer and make the LLVM optimizer more complex. So we have to detect the floating-point exceptions with additional code.

The first method for detecting floating-point exceptions is to check the exception flags in the underlying hardware with the `fetestexcept` function in the C++11 Standard Library. Because the exception flags in the underlying hardware may be changed inadvertently by the emulation manager, translator, or other modules in MC2LLVM or LLVM during execution, it is necessary to clear the exception flags with the `feclearexcept` function before executing every floating-point operation in order to avoid these outside disturbances, as shown in Figure 15.

```

// 61 == FE_ALL_EXCEPT indicates all exception flags
// clear all exception flags
%7 = call void @feclearexcept(i32 61)
// execute the body, say floating-point addition
%8 = fadd float %3, %6
// retrieve all exception flags
%9 = call i32 @fetestexcept(i32 61)
// check which exceptions occurs by testing %9

```

Figure 15. Use `feclearexcept` and `fetestexcept` to implement floating-point exceptions.

platform	execution time
MC2LLVM	67.72 seconds
QEMU	18.84 seconds

Figure 16. Execution time of one billion single-precision floating-point additions. Machine code to low-level virtual machine adopts the first method for emulating floating-point exceptions.

```

vaddss (%eax), %xmm0, %xmm0 // floating-point add, a
// floating-point exception may occur in this line
vmovss %xmm0, 8(%esp)
movl $61, (%esp)
calll 0x201DD07B // clear the exception flags
movl $61, (%esp)
calll 0x1FD8B04B // retrieve the exception flags

```

Figure 17. The erroneously re-ordered x86 code.

The first method comes with several drawbacks. First, MC2LLVM adopting this method becomes much slower than QEMU. We use a benchmark that executes one billion single-precision additions. The resulting execution time is shown in Figure 16. Furthermore, we wrote two new functions in x86 assembly to replace `fetestexcept` and `feclearexcept` to make emulation of floating-point operations even faster, but they are still a little (about 0.5 s) slower than QEMU evaluated by the benchmark. Second, the LLVM optimizer may reorder the instructions, which causes unexpected results, as shown in Figure 17, in which, the code erroneously clears exception flags *after*, rather than before, a floating-point operation.

The second method is to use additional code in LLVM IR to detect floating-point exceptions based on the operands and results of a floating-point operation.

Normally, floating-point exceptions are detected at various stages during the execution of a floating-point operation. For example, the occurrence of the underflow and inexact exceptions is determined based on the before-rounding intermediate values of a floating-point operation. However, we cannot obtain any intermediate values from an LLVM floating-point operation. We could reproduce the intermediate values with software, but this would incur high overhead. Alternatively, we use some tricks, which we will discuss in details later for better efficiency, for example, employing other floating-point operations to test the occurrence of floating-point exceptions and bypassing useless detection of floating-point exceptions. We will use floating-point additions and multiplications to illustrate this approach. Other floating-point operations (`vcvt`, `vdiv`, `vcge`, `vcmpe`, etc.) are handled similarly. MC2LLVM adopting this second method is twice as fast as QEMU according to our benchmarks (for floating-point additions) mentioned earlier. Therefore, we decide to use this second method in our binary translator. In what follows, we will discuss the emulation of each floating-point exception.

3.5.1. Emulation of the invalid operation exceptions. The occurrence of an invalid operation exception can be determined before executing a floating-point operation (but after the flush-input-to-zero processing). For example, detecting an invalid operation exception in floating-point addition is shown in Figure 18, where a positive infinity is added to a negative infinity or the addition involves one or more SNaNs.

```

1.  if((isInfinity(op1) && isInfinity(op2)
2.      && sign(op1) != sign(op2))
3.      || isNaN(op1) || isNaN(op2))
4.      SetI0C();

```

Figure 18. An algorithm for detecting the invalid operation exception in floating-point addition.

Table IV. Default results in case of an overflow exception.

Rounding mode	Sign	Default result for positive sign	Default result for negative sign
Round to nearest	(RN)	+infinity	-infinity
Round to +	(RP)	+infinity	-MaxNorm
Round to -	(RM)	+MaxNorm	-infinity
Round to 0	(RZ)	+MaxNorm	-MaxNorm

```

1. FP32Add_Overflow(op1, op2, sum)
2.  if(isInfinity(sum) || isMaxNorm(sum))
3.      if(!isInfinity(op1) && !isInfinity(op2))
4.          if(isInfinity(sum))
5.              SetOFC(); // set the overflow exception flag
6.      else if(sum - op2 != op1) // sum == MaxNorm
7.          SetOFC();

```

Figure 19. A *flawed* algorithm for detecting the overflow exception in single-precision floating-point additions.

3.5.2. Emulation of the division by zero exceptions. This case is similar to the emulation of the invalid operation exception. A division by zero exception occurs when a division operation has a zero divisor. This condition can be checked before executing the operation (but after the flush-input-to-zero processing).

3.5.3. Emulation of the overflow exceptions. The result of a floating-point operation can be a clue to determine if an overflow exception has occurred. In Neon/VFP, the default results in case of an overflow exception in different rounding modes are shown in Table IV, where *MaxNorm* is the maximum normalized number of the destination precision. An overflow exception occurs if the result of a floating-point operation is the (positive or negative) infinity or the (positive or negative) *MaxNorm*, depending on the rounding mode.

If an operation does not operate on one or more infinities but yields an infinity, the overflow exception has occurred. However, if the after-rounding result is *MaxNorm*, we cannot determine whether the before-rounding intermediate result is *MaxNorm*. In this case, we can reproduce the before-rounding intermediate result, although reproduction in software typically comes with high overhead, in order to determine if an overflow exception has occurred. Alternatively, we can apply the inverse operations (e.g., checking addition by subtraction) to test for overflow exceptions carefully. Figure 19 shows a flawed algorithm for detecting overflow exceptions for floating-point addition. This algorithm is flawed because it does not consider the limited precision (23 bits for single precision and 52 bits for double precision) of a floating-point number, that is, sum or $sum - op2$ may be inexact. For example, assume $op1$ is 1.0, $op2$ is $+MaxNorm$, and the rounding mode is RZ. Their sum is also $+MaxNorm$. In this case, $sum - op2$ is unequal to $op1$, but no overflow exception would occur.

Figure 20 shows a (corrected) algorithm for detecting overflow exceptions in single-precision floating-point additions regardless of the rounding mode. If sum is $+MaxNorm$, the difference $sum - big$ must be exact because big is greater than or equal to a half of $+MaxNorm$ (if big is less than a half of $+MaxNorm$, so is $small$; hence, sum cannot be $+MaxNorm$). That is, the exponent field of big is equal to that of $+MaxNorm$. Therefore, the subtraction operation $sum - big$ does not need to shift the radix point before subtraction (i.e., no bits will be lost) leading to an


```

1. FP32Add_Overflow(op1, op2, sum)
2.   if(isInfinity(sum) || isMaxNorm(sum))
3.     if(!isInfinity(op1) && !isInfinity(op2))
4.       if(isInfinity(sum))
5.         SetOFC(); // set the overflow exception flag
6.       else // sum == MaxNorm
7.         sum = abs(sum)
8.         op1 = abs(op1)
9.         op2 = abs(op2)
10.        big = max(op1, op2)
11.        small = min(op1, op2)
12.        remainder = (small - (sum - big))
13.        if(remainder >= 2104) SetOFC();

```

Figure 20. An algorithm for detecting the overflow exception in single-precision floating-point additions.

```

1. FPMul_Inexact(op1, op2, product)
2.   A = multiply op1's and op2's fractions with leading 1 or
   0;
3.   B = add op1's and op2's exponents;
4.   find the rounding point in A according to B;
5.   if there are non-zero bits to the right of the rounding
   point in A, then an inexact exception would occur;

```

Figure 21. An algorithm for detecting the inexact exceptions in floating-point multiplication.

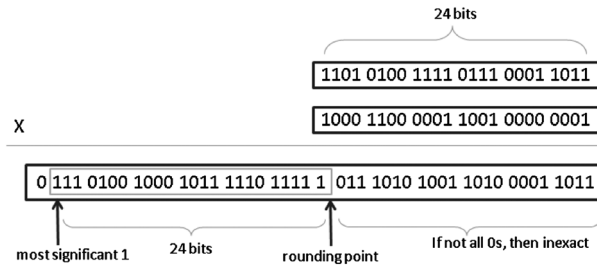


Figure 22. Multiplying two fractions in single-precision floating-point multiplication.

exact result. The magnitude of the least significant bit of the fraction of *MaxNorm* is 2^{104} . If the remainder is greater than or equal to 2^{104} , an overflow exception must have occurred because $\text{big} + \text{small}$ is greater than sum . On the other hand, if remainder is less than 2^{104} , the remainder is from a part of the fraction of small , which has been discarded in $\text{big} + \text{small}$ because of the rounding down operation. Hence, $\text{big} + \text{small}$ is indeed equal to $+\text{MaxNorm}$.

3.5.4. Emulation of the input denormal exceptions. This detection depends on the flush-input-to-zero processing. By definition, if any operand of an operation is flushed to zero, it causes an input denormal exception.

3.5.5. Emulation of the inexact exceptions. The inexact exception cannot be determined solely based on the final result of an operation. Because, unlike other floating-point exceptions, it may occur very frequently in usual floating-point calculations, the overhead of detecting the inexact exception should be reduced as low as possible.

Detecting inexact exceptions in floating-point multiplications in our implementation is implemented in software (by integer multiplication, addition, and shift). Figure 21 shows a simplified algorithm. A normalized number comes with an implicit leading 1 in the fraction part (the fraction part contains 23 bits in a single-precision floating-point number), but a denormalized number has an implicit leading 0. In the usual cases, the two operands' fractions are multiplied together (as integers) and then, in the single precision case, take the consecutive 24 bits starting from the leftmost 1 bit, as shown in Figure 22. The rounding point is at the immediate right of the 24 bits. For denormalized

option	execution time
IXC disabled	6.36 seconds
IXC enabled	14.31 seconds
QEMU	18.69 seconds

Figure 23. Execution tim of one billion single-precision multiplications.

```

1. FPMul_Inexact(op1, op2, product)
2.   if IXC == 1 and abs(product) > +MinNorm, then return;
3.   A = multiply op1's and op2's fractions with leading 1 or
   0;
4.   B = add op1's and op2's exponents;
5.   find the rounding point at A according to B;
6.   if there are non-zero bits to the right of the rounding
   point in A, then an inexact exception would occur;

```

Figure 24. A refined algorithm for detecting inexact exceptions in floating-point multiplications with a shortcut test.

option	execution time
no shortcut	14.31 seconds
shortcut + no IX occurs	15.48 seconds
shortcut + some IXs occur	7.37 seconds

Figure 25. Execution time of one billion single-precision floating-point multiplications with and without shortcut tests for detecting inexact exceptions.

numbers, the rounding point may be moved to the left because the exponent of the product is too small to be represented. The part on the right side of the rounding point will be discarded. The result is inexact if and only if any non-zero bits are discarded. This is a loss of precision, and an inexact exception has occurred.

Figure 23 shows the execution time of one billion single-precision multiplications. The logos `IXC disable` and `IXC enable` stand for MC2LLVM without and with the detection of the inexact exceptions, respectively. As you can see, the execution time when the detection is enabled is more than double the execution time when `IXC` is disabled. We further develop a shortcut that can avoid some of the useless detections of inexact exceptions.

Because the inexact cumulative exception flag, `IXC`, is *cumulative* (or sticky, i.e., the flag is not reset to 0 if a floating-point operation does not cause the inexact exception), it is to no avail to detect the inexact exception if `IXC` is already set to 1. Furthermore, notice that detecting the underflow exception (discussed later) requires the information whether an inexact exception has occurred. We add a shortcut that decides whether to skip the procedure for detecting the inexact exception, as shown in Figure 24, line 1. The detection of the inexact exception can be avoided under two conditions: (i) the `IXC` flag is already set to 1 and (ii) the underflow exception did not occur, that is, the absolute value of the before-rounding result of an operation is less than $+MinNorm$.[‡]

We measured the effects of avoiding the detections of the inexact exceptions. Figure 25 shows the execution time of one billion single-precision multiplications (in this experiment detecting `IXC` is enabled). The logo `no shortcut` stands for binary translation that uses Figure 21 for detecting inexact exceptions. The logo `shortcut + no IX occurs` stands for binary translation that uses Figure 24, but the inexact exceptions never occur. The logo `shortcut + some IXs occur` stands for binary translation that uses Figure 24, and some inexact exceptions indeed occur.

[‡]Because, by definition, the underflow exception occurs if the absolute value of the before-rounding result of an operation is less than $+MinNorm$ (which is the minimum normalized number of the destination precision) and the rounded result is inexact, the detection of the underflow exception relies on the result of detecting the inexact exception. We should ensure it is not possible to cause the underflow exception (i.e., the absolute value of a result of an operation is greater than $+MinNorm$) before bypassing the test.

```

1. FPAdd.Inexact(op1, op2, sum)
2.  if(IXC == 1) return; // shortcut
3.  condition1 = (sum - op1) != op2;
4.  condition2 = (sum - op2) != op1;
5.  if(condition1 || condition2) SetIXC();

```

Figure 26. An even faster algorithm for detecting the inexact exceptions in floating-point addition.

option	execution time
IXC disabled	6.53 seconds
no IX occurs	7.24 seconds
some IXs occur	6.57 seconds
QEMU	18.84 seconds

Figure 27. Execution time of one billion single-precision floating-point additions.

From Figure 25, the execution time is cut in half with the shortcut technique in Figure 24. The shortcut technique improves the performance because the inexact exceptions occur very often in most floating-point applications.

For floating-point additions, the shortcut can be implemented with an even faster trick, which is shown in Figure 26. Some detections of inexact exceptions in the floating-point operations are implemented in a tricky way. Figure 26 shows the detection of inexact exceptions in floating-point additions. Note that if a floating-point addition is not invertible, there must be a loss of precision during the addition. That is, if $(a + b) - a < b$, the sum $a + b$ must be inexact. In this case, an inexact exception should be raised.

Because addition can never cause the underflow exception, the shortcut test in floating-point addition, shown in Figure 26, is different from that test in floating-point multiplication, shown in Figure 24. Figure 27 shows the execution time of one billion single-precision floating-point additions. The logo `IXC disabled` stands for binary translation without the detection of inexact exceptions. The logo `no IX occurs` stands for one with the detection of inexact exceptions (using the faster algorithm in Figure 26), but inexact exceptions never occur. The logo `some IXs occur` stands for one with the detection of inexact exceptions (using the faster algorithm in Figure 26), and some inexact exceptions indeed occur. From Figure 27, we can see that our implementation of the detection of inexact exceptions in floating-point additions imposes little overhead.

3.5.6. Emulation of the underflow exceptions. An underflow exception is raised if the absolute value of the before-rounding intermediate result of a floating-point operation is less than $+MinNorm$ and the rounded result is inexact. The implementation is similar to the flush-output-to-zero procedure mentioned in Section 3.3. Note that the implementation relies on the result of detecting inexact exceptions.

Figure 28 summarizes the emulation of a floating-point operation. Note that not all floating-point operations will go through all the steps.

The emulation of floating-point operations involves many details and is error-prone for implementation. Therefore, a good verification technique is required for every implementation.

4. VERIFICATION OF THE BINARY TRANSLATOR

Testing is a crucial building block in order to achieve a high degree of reliability. It is difficult to identify the mistranslated instructions generated by a binary translator because there are so many translation functions in a binary translator and, due to the complex interdependencies inherent in a floating-point operation, it is almost impossible to determine whether an instruction is functionally equivalent to its translated instructions directly.

Mistranslated instructions may produce wrong values or fail to raise status flags such as the cumulative exception flags (IXC, UFC, etc.) and the cumulative saturation flag in FPSCR, in different

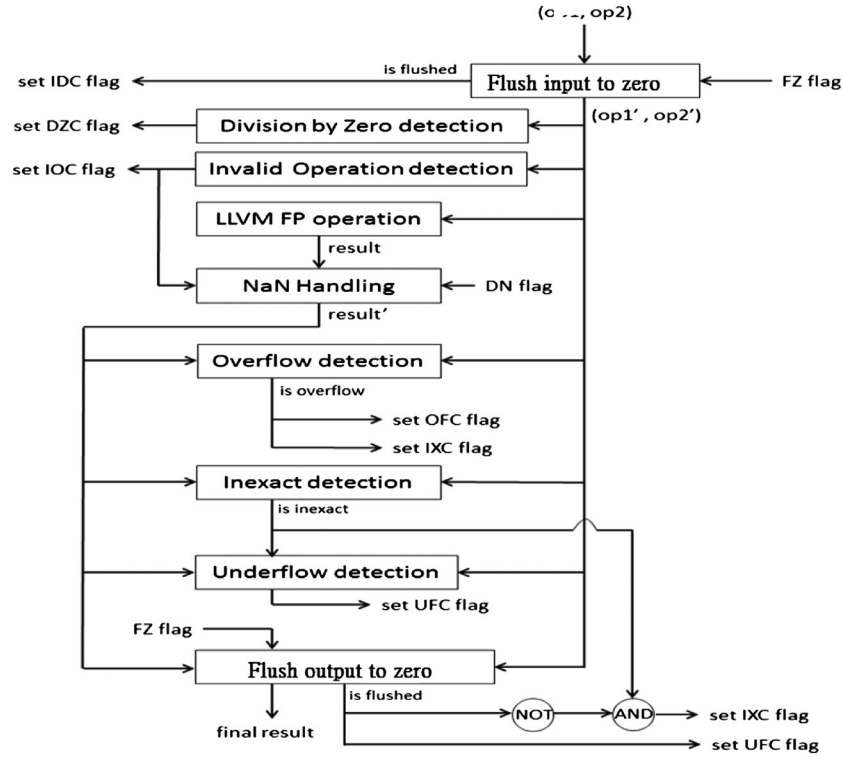


Figure 28. Emulation of a floating-point operation.

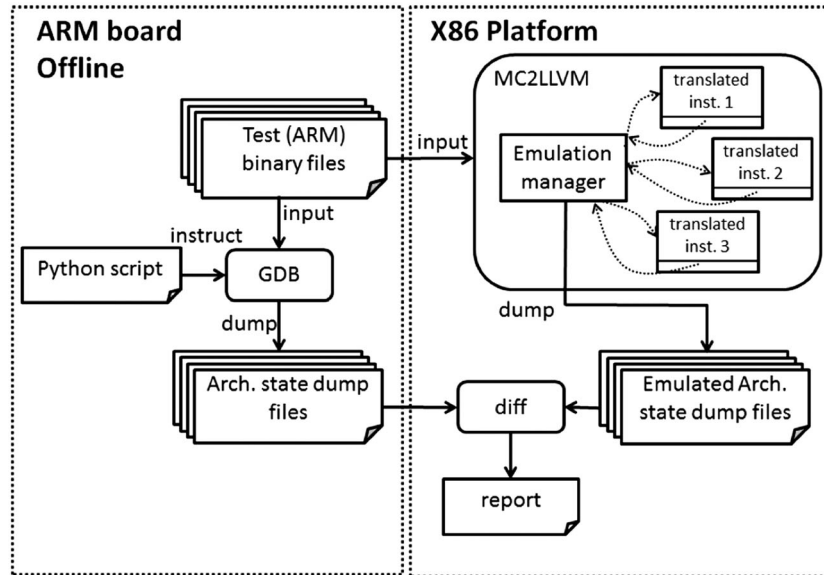


Figure 29. A verification framework.

combinations of modes, such as the flush-to-zero mode and the default NaN mode. Running a finite number of benchmarks correctly is far from being satisfactory for a binary translator because of low instruction coverage (an application usually makes use of less than 5% of the 1240 instructions in Neon and VFP), neglect of floating-point exceptions, and no mode switching in applications. Therefore, we developed a black-box testing framework for a binary translator, shown in Figure 29.

We execute ARM binary files with gdb [33] and a Python [34] script, which instructs gdb to dump the architecture state (ARM core registers and Neon and VFP registers, including FPSCR)

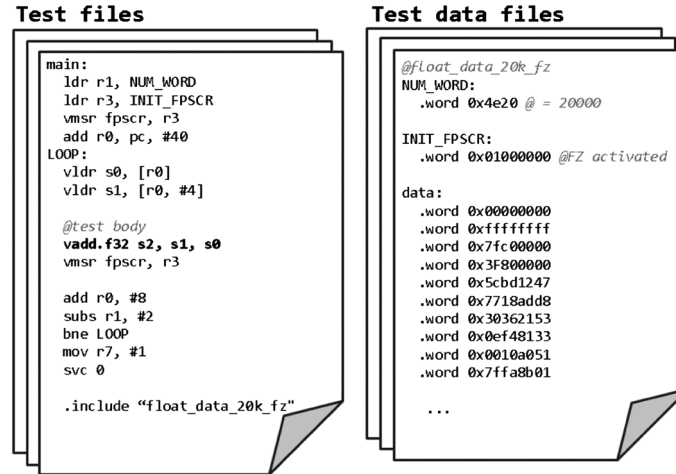


Figure 30. A test program and the accompanying test data.

after each ARM instruction is executed. The ARM instructions are executed on a cortex-A9-based PandaBoard ES (OMAP 4430) [35] as golden runs (i.e., fault-free runs). The architecture states from running the translated code generated by MC2LLVM are compared against that from the golden runs. MC2LLVM is slightly modified in order to dump the architecture state after each instruction is emulated.

For each Neon and VFP instruction, we wrote up a test program in assembly. The instruction is executed repeatedly for various combinations of operands. Figure 30 shows the test program (for the `vadd.f32` instruction) and the accompanying test data. The test program contains a loop. In each iteration of the loop, the `vldr` instructions load the operands from a test data file, and the instruction(s) under testing (`vadd.f32` in Figure 30) is performed. The loop may contain floating-point operation(s) or saturating instruction(s). The instruction `vmsr fpscr, r3` initializes the cumulative flags in FPSCR in order to catch floating-point exceptions and saturation and condition flags. The test data file also contains a word `INIT_FPSCR`, which determines the mode of execution.

A test data file is composed of two parts: a set of the predefined test cases and a set of randomly generated values over the appropriate range. There could be 10,000 or more test cases in a test data file. The predefined test cases include combinations of the extreme values, such as (positive and negative) maximum normalized numbers, (positive and negative) maximum denormalized numbers, (positive and negative) infinity, and (positive and negative) 0 for testing floating-point operations, and (positive and negative) 0, (positive and negative) 1, and the (positive and negative) maximum integer numbers for testing integer operations. Figure 31 illustrates the generation of the test data file. Executing each test program together with the accompanying test data takes a second on MC2LLVM and several minutes on the ARM board.

5. EXPERIMENTS

In order to show the performance of our translator, we compare the execution time of the guest binary with and without Neon and VFP instructions on MC2LLVM and QEMU, respectively.

Because QEMU is a dynamic translator, MC2LLVM also runs in the dynamic translation mode in these experiments.

5.1. Environment

The experimental hardware is equipped with Intel Core i7-4770 and 8-GB memory, running 32-bit Ubuntu version 14.04. We use SPEC CPU2006 [36] and LINPACK [37] as our benchmarks. All of the benchmarks were compiled into ARM statically linked binaries with GNU GCC version 4.4.6

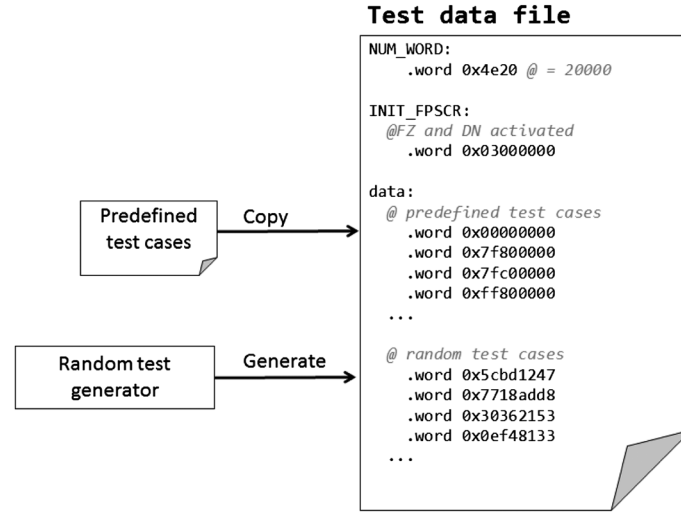
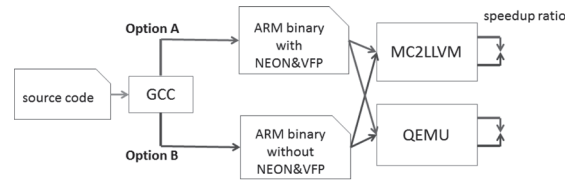


Figure 31. Generation of a test data file.



(a) Performance comparison for options A and B.



(b) Performance comparison for MC2LLVM and QEMU.

Figure 32. Performance comparison.

and linked with uClibc library 0.9.30.2 [38]. The version of QEMU we use is 2.1.50. MC2LLVM is layered on top of LLVM version 3.2.

We compiled SPEC CPU2006 with two configurations: Option A (-O3 -march=armv5 -mfloat-abi=softfp -mfpu=neon -ftree-vectorize) to tell the compiler to try to generate Neon and VFP instructions and Option B (-O3 -march=armv5 -mfloat-abi=soft -ftree-vectorize) to tell the compiler not to generate Neon and VFP instructions. The LINPACK benchmark was compiled with -O3 -march=armv5 -mfloat-abi=softfp -mfpu=vfpv3.

Among 12 integer benchmarks (CINT2006) in SPEC CPU2006, perlbench cannot be compiled and both MC2LLVM and QEMU failed to run the gcc benchmark in CINT2006. Among 17 floating-point benchmarks (CFP2006) in SPEC CPU2006, both MC2LLVM and QEMU failed to run gromacs, cactusADM, and sphinx3, and only MC2LLVM failed to run tonto and games. These failures are due to the big sizes of the benchmarks and probably bugs in MC2LLVM.

5.2. Performance

Figure 32(a) and (b) shows our comparative approaches. Figure 33 shows the execution time ratio of SPEC CINT2006 compiled with Option A and Option B running on MC2LLVM and QEMU, respectively. The geometric means of the execution time ratio for MC2LLVM and QEMU are 1.174 and 1.052, respectively.

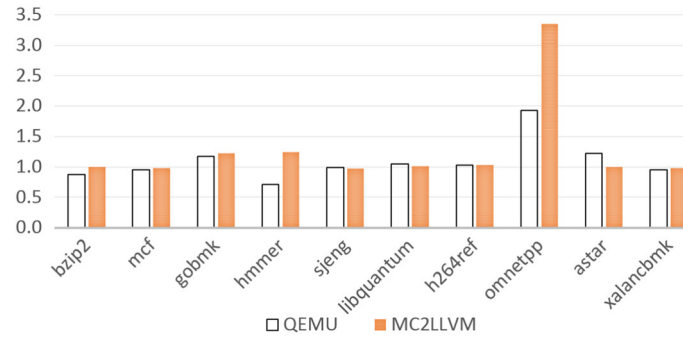
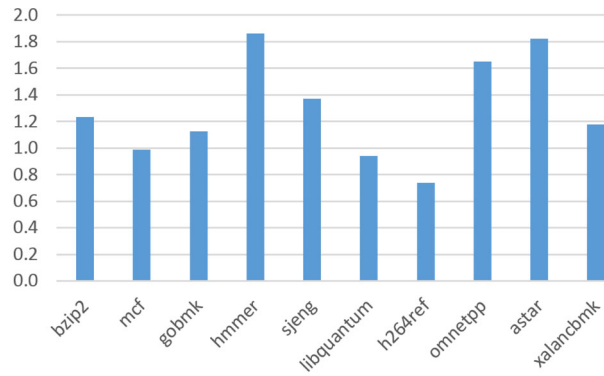
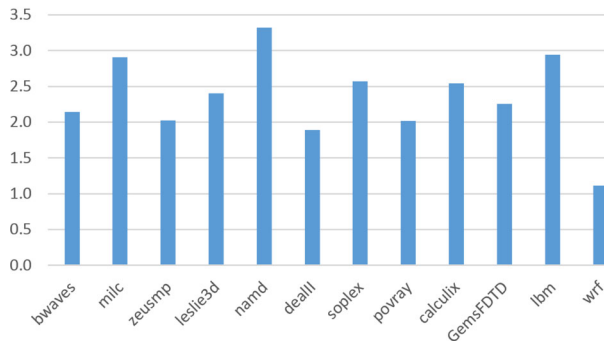


Figure 33. Execution time ratio of Option B/Option A on translating SPEC CINT2006.



(a) SPEC CINT2006.



(b) SPEC CFP2006.

Figure 34. Normalized speedup of machine code to low-level virtual machine over quick emulator.

The results imply that MC2LLVM could process the Neon and VFP instructions more effectively than QEMU on average.

Figure 34 shows the normalized speedup of MC2LLVM over QEMU on emulating SEPC CINT2006 and SPEC CFP2006, respectively. The geometric means are 1.24 and 2.27, respectively. The results are highly dependent on the SIMD and floating-point instructions generated by the compilers. It may be difficult to generate the Neon instructions for autovectorization in GCC from source code, but identifying floating-point operations is relatively easier. The fact can be observed from Table V, which shows the dynamic instruction counts of SPEC CPU2006 benchmarks. The Neon instructions generated by the compiler is too few to be perceptible. Moreover, Neon is designed for applications like multimedia algorithms. It provides many instructions for these purposes, for example, structure load/store instructions (for loading/storing RGB data structures), which were not abundant in these benchmarks. In contrast, for some benchmarks, especially CFP2006, the

Table V. Dynamic instruction counts of SPEC CPU2006 benchmarks.

Benchmark	ARM core										NEON and VFP			
	DP					NEON					NEON and VFP commons			
	L/S					VFP								
	DP	PC	Load	Br	Sys	Total	DP	L/S	strt	DP	L/S	Trans	Total	%nv
bzip2	1.2e12	1.8e6	1.0e12	1.1e7	1112	2.6e12	6.2e5	0	0	0	8.2e8	1.2e7	8.3e8	0.0
mcf	1.6e11	1.0e6	1.2e11	3.3e7	931	3.6e11	0	0	0	24	0	30	54	0.0
gobmk	9.8e11	1.1e9	7.0e11	1.4e10	8376	2.0e12	4	0	0	5.4e9	3.0e9	5.4e9	1.4e10	0.7
hmmer	1.5e12	2.0e6	1.9e12	3.4e9	1.3e4	3.5e12	7.0e5	0	0	7.6e9	4.2e9	3.9e9	1.5e10	0.4
sjeng	1.2e12	2.7e8	9.3e11	1.9e10	1.0e4	2.6e12	0	0	0	1.1e4	2.7e9	2.7e9	5.4e9	0.2
libquantum	1.9e12	285	5.5e11	2.7e8	174	2.8e12	0	0	0	8.6e8	3.0e9	1.9e8	4.1e9	0.1
h264ref	1.5e12	1.2e9	2.7e12	2.6e9	1.1e4	4.5e12	3.9e9	0	0	6.3e8	1.5e10	3.0e9	2.3e10	0.5
omnetpp	1.9e11	9.3e8	1.8e11	1.1e10	1.7e4	4.8e11	0	0	0	9.9e9	2.2e10	1.0e10	4.3e10	8.3
astar	6.0e11	1.9e5	3.9e11	5.1e9	1.2e5	1.1e12	0	0	0	1.8e8	3.7e7	1.7e8	4.0e8	0.0
xalancbmk	6.6e11	4.4e8	4.1e11	1.6e10	6.2e4	1.4e12	0	0	0	1.3e9	1.0e9	2.2e9	4.6e9	0.3
bwaves	1.7e12	7763	5.0e11	2.7e10	2307	2.5e12	0	0	0	2.9e11	5.3e11	1.0e10	8.3e11	24.6
milc	6.5e10	1.5e5	2.5e10	3.2e8	1.3e4	1.1e11	0	0	0	2.4e11	3.4e11	4.7e9	5.9e11	84.0
zeusmp	6.4e11	1.3e4	1.9e11	3.7e4	43	8.7e11	0	0	0	6.4e11	3.9e11	3.4e11	1.3e12	61.4
leslie3d	1.1e12	2.3e5	5.8e11	1.7e6	1.0e4	1.8e12	12	0	0	4.8e11	6.1e11	3.3e8	1.1e12	36.7
namd	4.3e11	6.6e6	2.0e11	1.2e9	2839	7.1e11	6.4e6	0	0	7.0e11	4.0e11	6.8e10	1.1e12	61.9
dealII	6.5e11	1.3e8	3.1e11	5.5e9	8.0e5	1.2e12	9.4e4	0	0	9.6e10	2.4e11	2.4e10	3.6e11	22.4
soplex	3.0e11	2.2e8	1.2e11	1.8e9	1.2e5	5.5e11	0	0	0	8.2e10	1.1e11	4.5e10	2.4e11	30.5
povray	3.9e11	1.9e8	2.0e11	2.3e10	2.7e4	7.5e11	1025	0	0	1.3e11	1.5e11	3.6e10	3.2e11	30.1
calculix	8.7e11	9.2e7	2.1e11	5.8e9	7.3e5	1.2e12	1.3e7	0	0	1.7e12	8.1e11	1.1e10	2.6e12	67.4
GemsFDTD	1.0e12	2.3e7	6.5e11	1.3e10	9.6e4	1.8e12	7.8e5	0	0	5.3e11	6.0e11	2.0e10	1.1e12	39.0
lbm	8.6e10	4e71	3.9e10	24225	664	1.4e11	0	0	0	5.8e11	2.1e11	1.1e11	9.1e11	86.4
wrf	8.6e12	9.8e6	8.2e11	2.2e11	9.9e5	1.1e13	8.5e9	0	0	7.3e11	7.5e11	1.2e11	1.6e11	12.8

DP, data-processing instruction counts; DP PC, data-processing-with-PC-as-destination-register instruction counts; L/S, load/store instruction counts; Br, branch instruction counts; Load PC, load-with-PC-as-destination-register instruction counts; Sys call, system-call counts; strt L/S, element-and-structure-load/store instruction counts; ARM Trans, data-transfer-between-ARM-core-and-extension-registers instruction counts; %nv, percentage of Neon and VFP instruction counts.

	MC2LLVM		QEMU	
	Option B	Option A	Option B	Option A
KFLOPS	12,876	156,062	12,044	46,431

Figure 35. The KFLOPS of Option B and Option A for LINPACK in machine code to low level virtual machine and quick emulator, respectively.

VFP instructions greatly contribute the speedup. Alternatively, we can say that VFP instructions considerably reduce the dynamic instruction counts of ARM core instructions because all floating-point calculations carried out with library functions were substituted with floating-point calculations carried out by the FPU.

Figure 35 shows the comparison of throughputs under similar setting (-mfpv=vfpv3 instead of -mfpv=neon in Option A) for the LINPACK benchmark. The matrix size is 200×200 .

It shows $12.12\times$ increase in throughput of the floating-point operations for MC2LLVM when VFP instructions are supported ($12.12 = 156062/12876$). Similar increase for QEMU is $3.85\times$ ($3.85 = 46431/12044$). Moreover, throughput for MC2LLVM is $3.36\times$ higher than that for QEMU on the LINPACK benchmark with VFP support ($3.36 = 156062/46431$). These results imply that MC2LLVM could process floating-point operations more efficiently than QEMU.

6. CONCLUSION

We implemented the Neon and VFP extensions in our binary translator MC2LLVM: 1240 translations of the Neon and VFPv3 instructions are emulated. Machine features of Neon and VFPv3 architectures, such as the condition flags, the flush-to-zero mode, the default NaN mode, and the floating-point exceptions, are included. MC2LLVM enhances the opportunity of using host SIMD and floating-point units to improve the performance of the translated programs. We also propose new methods (Section 3.5) by diagnosing the input and output of a floating-point operation in order to detect floating-point exceptions. This is useful when the IR layer of a binary translator does not provide the relevant information. Our method is much faster than pure software emulation. We also developed a verification framework for testing the emulated Neon/VFP instructions. The experimental results indicate that MC2LLVM is, in average, $1.24\times$ and $2.27\times$ faster than QEMU on the SPEC CPU2006 integer benchmarks and floating-point benchmarks, respectively, and have $3.36\times$ more throughput of the floating-point operations than QEMU benchmarked by LINPACK.

ACKNOWLEDGEMENT

The work reported in this paper is partially supported by Ministry of Science and Technology, Taiwan, Republic of China, under grants MOST 103-2221-E-009-105-MY3, NSC 100-2218-E-009-009-MY3, and NSC 100-2218-E-009-010-MY3.

REFERENCES

1. Sites RL, Chernoff A, Kirk MB, Marks MP, Robinson SG. Binary translation. *Communications of the ACM* 1993; **36**(2):69–81.
2. Chernoff A, Herdeg M, Hookway R, Reeve C, Rubin N, Tye T, Yadavalli SB, Yates J. FX!32 – a profile-directed binary translator. *IEEE Micro* 1998; **18**(2):56–64.
3. Zheng C, Thompson C. PA-RISC to IA-64: transparent execution, no recompilation. *Computer* 2000; **33**(3):47–52.
4. Chen J-Y, Yang W, Hung T-H, Su H-M, Hsu W-C. A static binary translator for efficient migration of ARM based applications. *Proceedings 6th Workshop on Optimizations for DSP and Embedded Systems*, Boston, Massachusetts, 2008; 55–64.
5. Shen B-Y, Chen J-Y, Hsu W-C, Yang W. LLBT: an LLVM-based static binary translator. *Proceedings 2012 International Conference on Compilers, Architectures and Synthesis for Embedded Systems*, (CASES' 12), Tampere, Finland, 2012; 51–60. DOI: 10.1145/2380403.2380419.

6. Shen BY, You JY, Yang W, Hsu W-C. An LLVM-based hybrid binary translation system. *Proceedings 7th IEEE International Symposium Industrial Embedded System (SIES 12)*, Karlsruhe, Germany, June 20–22, 2012.
7. Hazelwood K, Klauser A. A dynamic binary instrumentation engine for the ARM architecture. *Proceedings 2006 International Conference on Compilers, Architecture and Synthesis for Embedded Systems*, (CASES'06), Atlanta, Georgia, 2006; 261–270.
8. Hazelwood K, Lueck G, Cohn R. Scalable support for multithreaded applications on dynamic binary instrumentation systems. *Proceedings 2009 International Symposium Memory Management*, Dublin, Ireland, June 19–20, 2009; 20–29.
9. Luk C-K, Cohn R, Muth R, Patil H, Klauser A, Lowney G, Wallace S, Reddi VJ, Hazelwood K. Pin: building customized program analysis tools with dynamic instrumentation. *ACM SIGPLAN Notices* 2005; **40**(6):190–200.
10. Bala V, Duesterwald E, Banerjia S. Dynamo: a transparent dynamic optimization system. *ACM SIGPLAN Notices* 2000; **35**(5):1–12.
11. Bruening D, Garnett T, Amarasinghe S. An infrastructure for adaptive dynamic optimization. *Proceedings International Symposium Code Generation and Optimization (CGO 03)*, San Francisco, California, 2003; 265–275.
12. Lu J, Chen H, Yew P-C, Hsu W-C. Design and implementation of a lightweight dynamic optimization system. *Journal of Instruction-Level Parallelism* 2004; **6**(4):332–341.
13. Moore RW, Baiocchi JA, Childers BR, Davidson JW, Hiser JD. Addressing the challenges of DBT for the ARM architecture. *Proceedings 2009 ACM SIGPLAN/SIGBED Conference Languages, Compilers, and Tools for Embedded Systems (LCTES 09)*, Dublin, Ireland, 2009; 147–156. DOI: 10.1145/1542452.1542472.
14. Payer M, Gross TR. Fine-grained user-space security through virtualization. *Proceedings 17th ACM SIGPLAN/SIGOPS International Conference Virtual Execution Environments*, Vol. 46, ACM SIGPLAN Notices, Newport Beach, California, July 2011; 157–168.
15. Smith JE, Nair R. *Virtual Machines: Versatile Platforms for Systems and Processes* Morgan Kaufmann, 2005.
16. Cmelik B, Keppel D. Shade: a fast instruction-set simulator for execution profiling. *Proceedings 1994 ACM SIGMETRICS Conference Measurement and Modeling of Computer Systems*, Nashville, Tennessee, 1994; 128–137.
17. Barham P, Dragovic B, Fraser K, Hand S, Harris T, Ho A, Neugebauer R, Pratt I, Warfield A. Xen and the art of virtualization. *ACM SIGOPS Operating Systems Review* 2003; **37**(5):164–177.
18. VMware Inc. *VMware Workstation*, 2013.
19. Kivity A, Kamay Y, Laor D, Lublin U, Liguori A. Kvm: the linux virtual machine monitor. *Proceedings Linux Symposium* 2007; **15**:225–230.
20. ARM. *ARM Architecture Reference Manual ARMv7-A and ARMv7-R edition Errata Markup*, ARM DDI 0406B, 2011.
21. Bellard F. QEMU, a fast and portable dynamic translator. *Proceedings 2005 USENIX Annual Technical Conference*, Anaheim, California, 2005; 41–46.
22. Lattner CA. LLVM: an infrastructure for multi-stage optimization. *Master's Thesis*, Comp. Sci. Dept, Univ. Illinois at Urbana-Champaign, 2002.
23. Cytron R, Ferrante J, Rosen BK, Wegman MN, Zadeck FK. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 1991; **13**(4):451–490.
24. Firasta N, Buxton M, Jinbo P, Nasri K, Kuo S. Intel AVX: new frontiers in performance improvements and energy efficiency, Intel White Paper, 2008.
25. IEEE. *IEEE Standard for Floating-point Arithmetic*, 2008.
26. Jantz MR, Kulkarni PA. Analyzing and addressing false interactions during compiler optimization phase ordering. *Software-Practice and Experience* 2014; **44**(6):643–679. DOI: 10.1002/spe.2176.
27. Pryanishnikov I, Kralll A, Horspool N. Compiler optimizations for processors with SIMD instructions. *Software-Practice and Experience* 2007; **37**(1):93–113. DOI: 10.1002/spe.751.
28. Kretz M, Lindenstruth V. Vc: a C++ library for explicit vectorization. *Software-Practice and Experience* 2012; **42**(11):1409–1430. DOI: 10.1002/spe.1149.
29. Lemirel D, Boytsov L. Decoding billions of integers per second through vectorization. *Software-Practice and Experience* 2015; **45**:1–29. DOI: 10.1002/spe.2203.
30. Inoue H, Moriyama T, Komatsu H, Nakatani T. A high-performance sorting algorithm for multicore single-instruction multiple-data processors. *Software-Practice and Experience* 2012; **42**(6):753–777. DOI: 10.1002/spe.1102.
31. MCDisassembler. Available at: http://www.llvm.org/docs/doxygen/html/classllvm_1_1MCDisassembler.html [last accessed February 2016].
32. Schwarz EM, Schmookler M, Trong SD. Hardware implementations of denormalized numbers. *Proceedings 16th IEEE Symposium Computer Arithmetic*, (ARITH03), 2005.
33. Stallman RM, Pesch R. *Debugging with GDB: The GNU Source-Level Debugger*, Free Software Foundation, 2002.
34. van Rossum G, Drake Jr. FL. *The python library reference – release 2.7.8*, Python Software Foundation, 2014.
35. Texas Instruments. *Pandaboard, OMAP4430 SoC dev. board, revision A2* 2012.
36. Henning JL. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News* 2006; **34**(4): 1–17.
37. Dongarra JJ, Luszczek P, Petitet A. The LINPACK benchmark: past, present, and future. *Concurrency and Computation: Practice and Experience* 2003; **15**(9):803–820.
38. uClibc. Available at: <http://www.uclibc.org/> [last accessed February 2016].